🏠 HOUSE HUNT

MERN Stack Based House Rental Web Application

---

1. ABSTRACT

House Hunt is a full-stack web application developed using the MERN stack (MongoDB, Express.js, React.js, and Node.js) to provide a modern and efficient solution for property rental and listing management. The platform is designed to connect property owners and tenants through a secure and user-friendly online system.

The application allows users to register, log in securely, browse available properties, view detailed listings, and book rental houses. Property owners can list new properties, update details, and manage bookings. An admin module ensures system integrity by monitoring users, properties, and bookings.

The backend is built using Node.js and Express.js to handle RESTful APIs and business logic, while MongoDB is used to store user, property, and booking data. Secure authentication is implemented using JSON Web Tokens (JWT) and password encryption with bcrypt to ensure data protection and authorized access.

The frontend, developed using React.js, provides an interactive and responsive user interface that enhances user experience.

House Hunt digitizes the traditional property rental process, reduces dependency on brokers, and provides a scalable and secure solution for modern real estate management. The project demonstrates practical implementation of full-stack development concepts, database integration, authentication mechanisms, and real-world web application architecture.

The system allows users to:

- Register and log in securely
- Search for rental or sale properties
- View detailed property information
- Book rental properties
- List new properties

The application uses secure authentication with JWT and password encryption using bcrypt. MongoDB is used to store user data, property details, and booking records.

House Hunt simplifies the traditional house rental process by digitizing property management and eliminating middle agents.

---

2. OBJECTIVE

The primary objective of the House Hunt project is to design and develop a full-stack web application that provides a digital platform for managing house rentals and property listings efficiently using the MERN stack.

Main Objectives:

- To develop a user-friendly web application for property rental and listing.
- To implement secure user authentication and authorization using JWT.
- To create RESTful APIs for handling user, property, and booking data.
- To design and integrate a MongoDB database for storing structured information.
- To enable property owners to add, update, and delete property listings.
- To allow tenants to search, view, and book properties easily.
- To implement role-based access control for User, Owner, and Admin.
- To ensure data security through password encryption using bcrypt.
- To demonstrate full-stack development skills using modern technologies.
- To simulate a real-world real estate platform similar to commercial property websites.

Technical Objectives:

- To perform CRUD (Create, Read, Update, Delete) operations using Express APIs.
- To connect frontend and backend using Axios for API communication.
- To structure the project using modular architecture.
- To test APIs using Postman or Thunder Client.

Specific Goals:

- Allow owners to list properties easily
- Enable tenants to search and book properties
- Implement role-based access (User, Owner, Admin)
- Store data securely using MongoDB
- Provide responsive UI using React

3. TECHNOLOGY DESCRIPTION

3.1 Frontend – React.js

The frontend of the House Hunt application is developed using **React.js**, a popular JavaScript library for building modern and interactive user interfaces. React enables the creation of reusable components and ensures a smooth user experience through fast rendering.

React follows a component-based architecture, which makes the application modular, maintainable, and scalable.

Features:

- Component-based architecture
- Fast rendering using Virtual DOM
- Reusable UI components
- Smooth navigation using React Router

Responsibilities:

- Display property cards
- Show forms for login and listing
- Send API requests to backend
- Handle user sessions

3.2 Backend – Node.js & Express.js

The backend of the House Hunt application is developed using **Node.js** and **Express.js**. It is responsible for handling business logic, managing data operations, processing client requests, and ensuring secure communication between the frontend and the database.

Node.js provides the runtime environment, while Express.js is used to create RESTful APIs and manage server-side routing.

Backend Responsibilities:

- Handle HTTP requests
- Process business logic
- Authenticate users
- Connect to MongoDB
- Perform CRUD operations

3.3 Database – MongoDB

The House Hunt application uses **MongoDB** as its primary database system. MongoDB is a NoSQL database that stores data in a flexible, JSON-like format called BSON (Binary JSON). It is highly scalable, efficient, and well-suited for modern web applications built with Node.js.

MongoDB is integrated with the backend using **Mongoose**, which is an Object Data Modeling (ODM) library for Node.js.

Collections Used:

- Users
- Properties
- Bookings

Advantages:

- Flexible schema
- High performance
- Easy integration with Node.js

---

4. DETAILED DESCRIPTION OF PROJECT PHASES

Phase 1: Requirement Analysis

Requirement Analysis is the first and most important phase of the project development process. In this phase, the needs and expectations of users are identified and documented clearly.

For the House Hunt project, requirement analysis was done to understand what features the system should provide and how it should function.

Identified Requirements:

- User registration and login
- Property listing system
- Booking system
- Admin dashboard
- Secure authentication

Phase 2: System Design

System Design is the phase where the overall structure of the application is planned. It defines how different components of the system interact with each other, how data flows through the system, and how the application architecture is structured.

The House Hunt application is designed using a structured and modular approach based on the MERN stack architecture.

The application follows 3-tier architecture:

- Presentation Layer (React)
- Application Layer (Express)
- Data Layer (MongoDB)

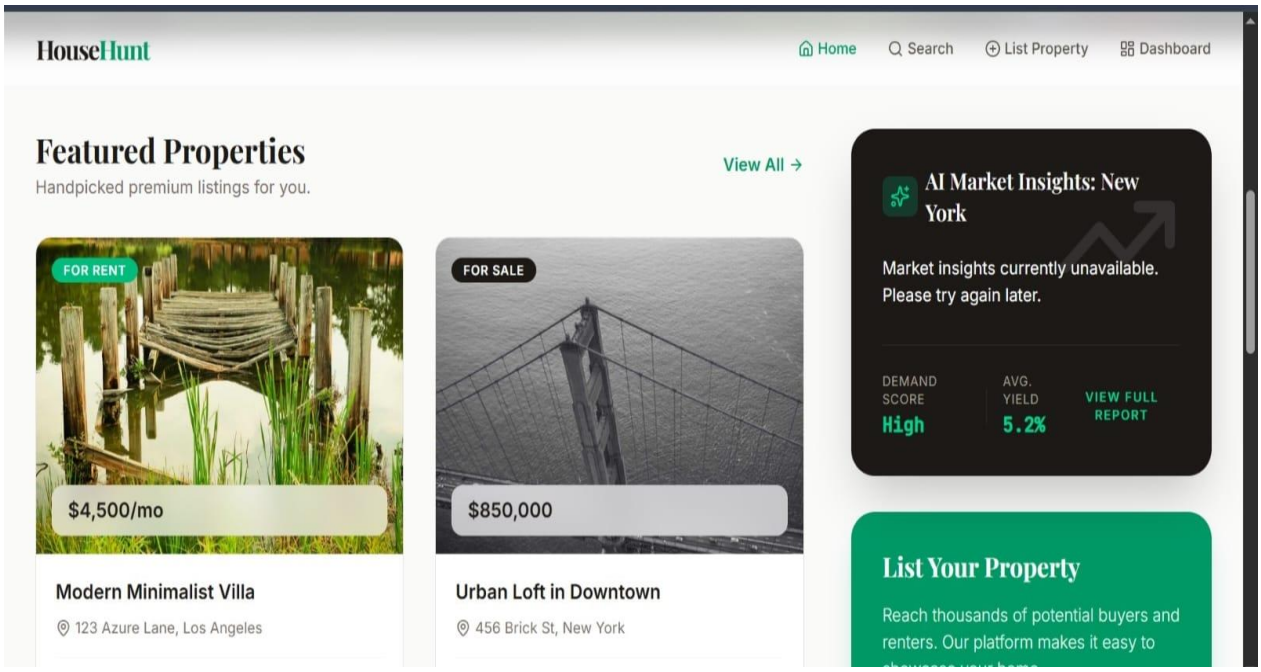
Database schema was designed for:

- Users
- Properties
- Bookings

Phase 3: Frontend Development

- Home Page

Description:



The homepage displays featured properties with details such as:

- Property image
- Price
- Location
- Number of bedrooms

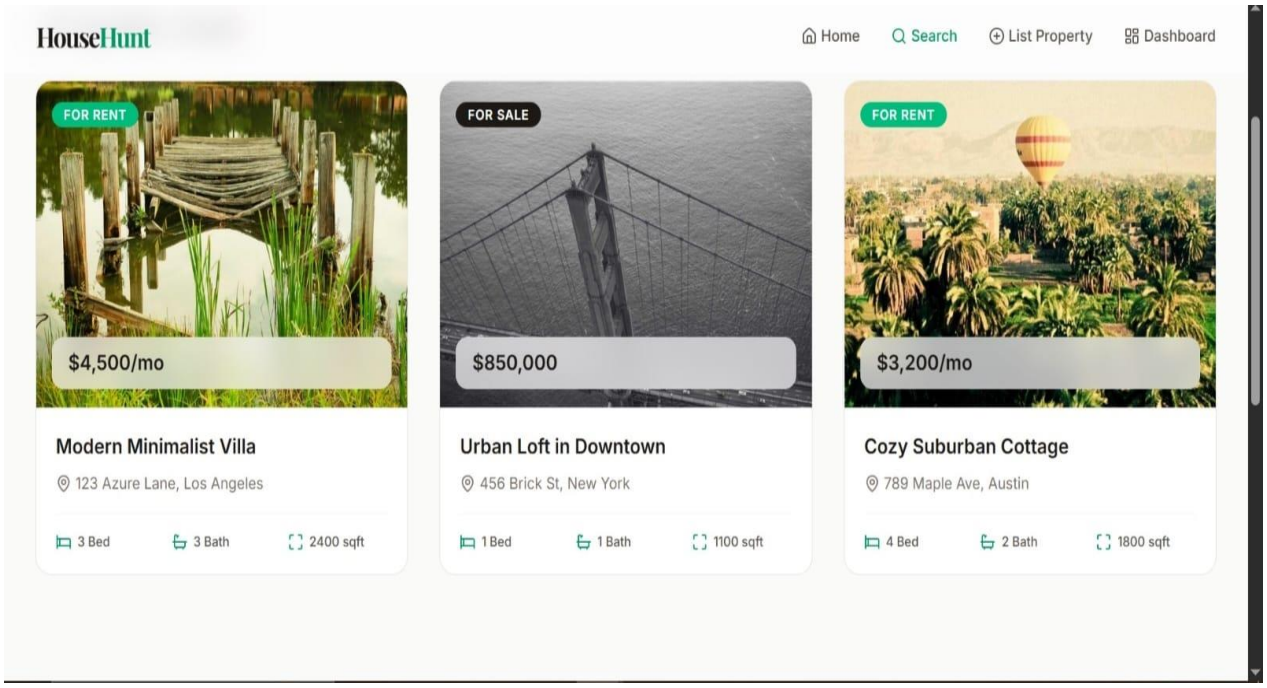
Features visible in screenshot:

- Navigation bar (Home, Search, List Property, Dashboard)
- Featured Properties section
- Property cards with rental/sale tags

Purpose:

- Provides users a quick overview of available properties
- Makes browsing simple and attractive

– Featured Properties & AI Market Insight



Description:

This section shows:

- Featured premium listings
- AI Market Insights card
- Demand score and average yield

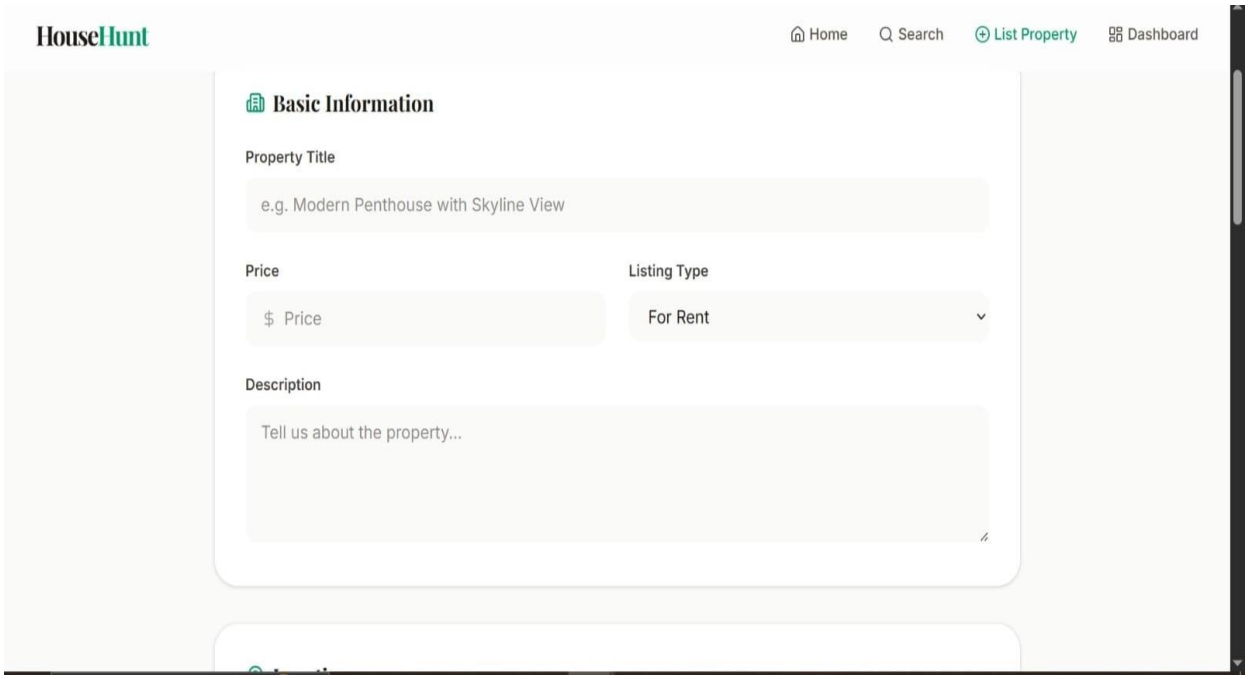
Features:

- Professional UI layout
- Highlighted market analysis section
- “View All” button for exploring more properties

Purpose:

- Provide advanced market information
- Improve user decision-making

– List Property Page



Description:

The **List Property Page** is one of the most important features of the House Hunt application. This page allows property owners to add new property listings to the platform. It acts as the interface between property owners and the system database.

This feature ensures that owners can easily publish their property details so that tenants can view and book them.

This page allows property owners to add new listings.

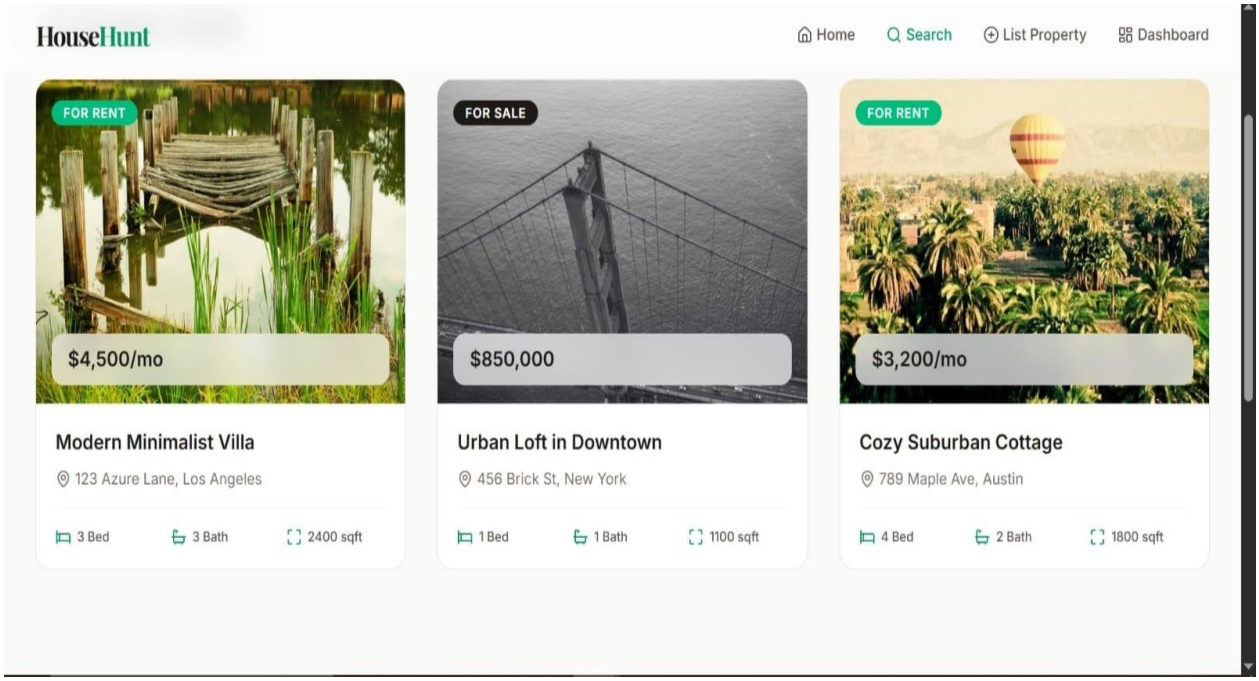
Fields include:

- Property Title
- Price
- Listing Type (For Rent / For Sale)
- Description

Purpose:

- Allows owners to submit property details
- Sends data to backend API
- Stores property in MongoD

– Why HouseHunt Section



Description:

This section highlights platform advantages:

- Verified premium listings
- AI-powered market analysis
- Direct communication
- Virtual tour capabilities

Purpose:

- Build user trust
- Explain platform benefits

Phase 4: Backend Development

APIs were created for:

- User registration
- Login authentication
- Property CRUD operations
- Booking system
- Admin control

Security implemented using:

- JWT authentication
- Password hashing (bcrypt)
- Protected routes

Phase 5: Database Integration

MongoDB collections were created for:

Users Collection:

- name
- email
- password
- role

Properties Collection:

- title
- description
- price
- location
- ownerId

Bookings Collection:

- userId
- propertyId
- bookingDate
- status

Mongoose was used to define schemas and models.

Phase 6: Testing

Testing tools used:

- Postman
- Thunder Client

Tested functionalities:

- User registration
- Login authentication
- Property creation
- Booking system
- API responses

---

5. PROJECT EXPLANATION WITH REAL-WORLD EXAMPLES

House Hunt works similar to real-world platforms like:

- Zillow
- 99acres
- MagicBricks

Example 1 – Tenant Use Case

This use case explains how a tenant (user) interacts with the House Hunt system to find and book a rental property.

Scenario

A student is moving to a new city for higher studies and needs to find a rental apartment within a specific budget and location.

Step-by-Step Process

1. User Registration
    - The student visits the House Hunt website.
    - Registers using name, email, and password.
    - Account is created securely with encrypted password.
2. Login
    - The student logs into the system using registered credentials.
    - The system verifies login details and generates a JWT token.
3. Search for Properties
    - The tenant browses the homepage.
    - Views featured properties.
    - Filters properties based on:
        - Location
        - Price range
        - Number of bedrooms
4. View Property Details
    - Clicks on a specific property.
    - Views detailed information such as:
        - Property description
        - Price
        - Area (sq ft)
        - Bedrooms and bathrooms
        - Images

5. Book Property
    - If satisfied, the tenant clicks on the “Book Now” option.
    - Booking request is sent to backend.
    - Booking is stored in MongoDB database.
6. Booking Confirmation
    - The system confirms booking.
    - Tenant can view booking history in dashboard.

Example 2 – Property Owner Use Case

This use case explains how a property owner interacts with the House Hunt system to list and manage rental properties.

Scenario

A homeowner wants to rent out their apartment to generate monthly income. Instead of contacting brokers, the owner uses the House Hunt platform to list the property online.

Step-by-Step Process

1. Owner Registration

- The owner visits the House Hunt website.
- Registers by entering name, email, password, and selecting role as “Owner”.
- The system encrypts the password and stores the data securely in MongoDB.

2. Login

- The owner logs in using registered credentials.
- Backend verifies details and generates a JWT token.
- Owner is redirected to dashboard.

3. Add New Property

- Owner navigates to the **List Property** page.
- Fills out the property form including:

    - Property Title
    - Price
    - Location
    - Description
    - Number of bedrooms
    - Number of bathrooms
    - Area
    - Image URL
- Clicks “Submit” button.
- Frontend sends POST request to backend API.
- Backend verifies owner role and saves property in database.

4. Property Display

- The newly added property appears on the homepage.
- Tenants can now view and book it.

5. Manage Property

The owner can:

- Edit property details
- Update price or description
- Delete property listing
- View booking requests

All updates are reflected in real time through database updates.

---

6. CONCLUSION

House Hunt is a complete MERN stack web application that successfully integrates frontend, backend, and database systems. The House Hunt project successfully demonstrates the development of a full-stack web application using the MERN stack (MongoDB, Express.js, React.js, and Node.js). The system provides a modern, secure, and user-friendly platform for managing property rentals and listings digitally.

The application effectively integrates frontend, backend, and database technologies to create a seamless experience for tenants, property owners, and administrators. Secure authentication using JWT and password encryption with bcrypt ensures data protection and controlled access to system features.

The project demonstrates:

- Full-stack development
- REST API implementation
- Secure authentication
- MongoDB data management
- Real-world rental platform logic

It provides a scalable and secure solution for property rental management.

The project enhances understanding of modern web development practices and prepares for real-world industry applications.

House Hunt simplifies the traditional property rental process by eliminating middle agents and providing centralized digital management. It demonstrates practical knowledge of full-stack development, database design, API integration, and system architecture.

Overall, the project provides a scalable foundation that can be enhanced in the future with features such as payment gateway integration, real-time chat, advanced search filters, and mobile application support.

The successful completion of this project enhances technical skills and provides real-world experience in developing modern web applications.